

通行止め・通行困難地点 更新・公開 運用手順書

バージョン: 1.0 **最終更新日:** 2026年7月16日 **対象:** 運用担当者（開発担当でない、通行止め情報を登録・公開する人） **関連:** [設計書](#) `closures-design-202607.md`

1. この手順書について

ハイキングマップ上に表示する「通行止め・通行困難地点」を**追加・修正・解除**し、ユーザーへ**公開**するための手順をまとめたものです。

2026年7月の改修により、公開方式が **公開API（Vercel Function + Vercel Blob）** に変わりました。これにより **PC・git は不要**で、**スマートフォン／タブレットのブラウザ**だけで、編集パネルの「公開」ボタン一つでユーザーへ即時公開できます。

公開が失敗したときの**原因の見分け方（エラーコード E01～E05）**と対応は [9章](#)にまとめています。

2. 事前準備（初回に一度だけ）

以下がそろっていることを確認してください。

準備するもの	内容
公開トークン	Vercel の環境変数 <code>CLOSURES_PUBLISH_TOKEN</code> に設定された秘密の文字列。運用担当者だけが知っておく（決め方・Vercel設定は 12章 ）
地点データ作成アプリ	通行止め地点の geojson を作成する別アプリ（MapGPS → MapEditor）
起動用リンク	MapGPS から本アプリを <code>?closure=true</code> 付きで開くリンク（このとき「通行止め・通行困難地点」ボタンが表示される）

公開トークンは**初回の「公開」時に一度だけ入力**します。以降はその端末に保存され、再入力は不要です。（端末を変えた場合や、間違えた場合は再入力になります → [8章](#)）

3. 扱うデータの形式

- 通行止め地点は **GeoJSON（FeatureCollection）** で、各地点は **Point（点）** です。
- データ本体は**別アプリ（MapGPS → MapEditor）**で作成します。本アプリでは作成せず、読み込み・確認・公開のみ行います。
- 公開時に本アプリ／API が確認する項目は次のとおりです。満たさないと公開できません（エラー表示）。

項目	必須	内容
<code>version</code> （先頭）	✓	データのバージョン。 更新のたびに必ず新しい値へ変える （→ 6章 ）
各地点が <code>Point</code>	✓	点データであること

項目	必須	内容
座標が箕面エリア内	✓	おおよそ経度 135.2～135.8 / 緯度 34.6～35.1 の範囲
id の重複なし	✓	id を付ける場合は全地点で一意

各地点で地図の見た目・ポップアップに使われる主なプロパティ:

プロパティ	説明	表示
kind	closed (通行止め) / difficult (通行困難)	マーカーの形・色 (通行止め=赤ひし形/通行困難=橙三角)
name	地点名 (例「風呂谷 倒木」)	ポップアップの見出し (無ければ id)
reason	理由 (工事/倒木/落石 など)	ポップアップに「理由: ...」
note	補足 (例「巻き道あり」)	ポップアップに表示
updatedAt	更新日 (YYYY-MM-DD 等)	ポップアップに「更新日: ...」

記入例など詳しいスキーマは[設計書 §4.2](#)を参照してください。

4. 更新の全体フロー

```

[別アプリ] 通行止め地点の geojson を作成/修正
    |
    ▼
[本アプリを ?closure=true で起動] → ホーム「通行止め・通行困難地点」
    |
    ▼
① ファイル読み込み → 地図で内容を確認 (この時点では未反映)
② バージョンを変更 → 「マップに反映」 (自分の端末だけに反映)
③ 「公開」 → ユーザーへ即時公開 (初回のみトークン入力)

```

「マップに反映」は自分の端末だけ / 「公開」で全員へ、という2段階構えです。

5. 手順 (通常の更新)

手順1. データを用意する

別アプリ (MapGPS → MapEditor) で、今回の通行止め地点を反映した **geojson ファイル** を用意します。

手順2. 本アプリを起動する

MapGPS から **?closure=true** 付きで**本アプリを起動**します。ホーム画面に「通行止め・通行困難地点」ボタンが表示されるので、これを押します。(マップ画面に切り替わり、右上に**編集パネル**が開きます)

手順3. ファイルを読み込んで確認する

編集パネルの「ファイル読み込み」を押し、手順1の geojson を選びます。地図に地点がプレビュー表示されるので、位置・種別・件数が正しいか確認します。

この時点ではまだ誰にも公開されていません（読み込んで表示しているだけ）。

手順4. バージョンを変更して「マップに反映」

編集パネルの「バージョン」欄を新しい値に変更します（例: 2026-07-16.1）。現在の値と違う値にすると「マップに反映」ボタンが押せるようになります。「マップに反映」を押し、確認ダイアログで OK すると、この端末のマップにだけ反映されます。

手順5. 「公開」でユーザーへ

編集パネルの「公開」を押します。

- 確認ダイアログで OK します。
- 初回のみ、公開トークンの入力を求められます。入力すると以降は保存され再入力不要です。
- 成功すると「ユーザーへ公開しました」と表示され、編集パネルが閉じます。

以上で公開完了です。他ユーザーの端末には、次回マップを開いたときに新しい内容が反映されます。

6. 地点の解除（削除）・全削除

公開は**「全置換」**です（前回分に追記ではなく、送ったファイルで丸ごと置き換え）。

- 一部の地点を解除したい → その地点を含まない geojson を作り、5章の手順で公開する。消えます。
- すべて解除したい → **0件（空）**の geojson を公開する。
 - このとき「0 件のため、公開中の全地点が地図から消えます」という警告付き確認が出ます。意図どおりなら OK します。

解除も「公開」操作で行います。別途の削除操作はありません。

7. バージョンの付け方

- 更新のたびに必ず新しい値に変える必要があります。バージョンは各端末の「新しいデータが来た」判定に使われ、同じ値だと更新に気づけません。
- 推奨形式は 日付.連番（例: 2026-07-16.1、同日中の2回目なら 2026-07-16.2）。既存データもこの形式です。
- 「マップに反映」は、バージョンを現在値から変更しないと押せません（変更忘れ防止）。

8. 公開トークンについて

- 公開トークンは Vercel の環境変数 CLOSURES_PUBLISH_TOKEN と同じ値です。
- 初回の「公開」時に一度入力すると、その端末に保存され、以降は再入力不要です。
- アプリのコードには埋め込まれていません（運用担当者だけが知っている前提）。
- トークンが違っていると「【E01】公開トークンが正しくありません」と表示され、保存が消えます。もう一度「公開」を押して正しい値を入力してください（→ 9章）。
- 別の端末で公開するときは、その端末で改めて入力が必要です。

トークンをどう決め、Vercel にどう設定・更新するか（初回セットアップ担当者向け）は [12章 付録](#) を参照してください。

9. 公開に失敗したときの対応（エラーコード別）

「公開」に失敗すると、画面に **エラーコード（E01～E05）付きのメッセージ** が出ます。運用担当者は、表示されたコードと文面をそのまま開発担当者に伝えれば、原因を切り分けられます。まず運用担当者自身で試せることを行い、直らなければ開発担当者へ連絡してください。

開発担当者の方へ: 運用担当者から「E〇〇が出た」と連絡を受けたら、下表の右2列で「担当者に案内すること」と「サーバー側でやること」がすぐ分かります。

9.1 コード早見表

コード	画面の要点	主な原因	運用担当者がまず試すこと	開発担当者の対応（サーバー側）
E01	公開トークンが正しくありません	入力したトークンが違う	もう一度「公開」→正しいトークンを入力（→ 8章 ）	値が不明なら正しい <code>CLOSURES_PUBLISH_TOKEN</code> を伝える／必要ならローテーション（→ 12.3 ）
E02	公開機能が設定されていません	サーバーに公開トークンが未設定、または設定後に未反映	（操作では直らない）開発担当者へ連絡	<code>CLOSURES_PUBLISH_TOKEN</code> を設定し再デプロイ（→ 12.2 ）
E03	公開データに不備があります	送信データ（geojson）の中身が不正	表示の「理由」を見てデータを直し、再公開（→ 下記 9.2）	通常は不要（データ側の問題）
E04	公開データの保存に失敗しました	公開ストア（Blob）への保存失敗	少し時間をおいて再試行	Blob 接続と <code>BLOB_READ_WRITE_TOKEN</code> を確認して再デプロイ、ログで生エラー確認（→ 下記 9.3）
E05	サーバーに接続できません（通信エラー）	通信断・接続不良・サーバー障害	電波／Wi-Fi を確認して再試行	Vercel の稼働・公開API（ <code>/api/closures</code> ）の到達性を確認

失敗時に「今回のデータをこの端末に保存しますか?」と出たら、**保存しておく**と安全です。あとで公開をやり直せるほか、開発担当者に渡して調べてもらえます（PC・git 作業は不要です）。

9.2 E03（データ不備）の代表例と直し方

E03 は**トークンではなくデータの中身**の問題です。画面の「理由:」を見て直します。

「理由」の表示	意味	直し方
version がありません	バージョンが未設定・空	「バージョン」を新しい値に変える (→ 7章)
...の座標が算面エリアの範囲外です	地点の座標が想定範囲外 (経度 135.2~135.8 / 緯度 34.6~35.1)	データ作成アプリ側で座標を修正
id が重複しています: ...	同じ id の地点が複数ある	重複した id を一意に直す
FeatureCollection / Point 形式ではありません	ファイル構造が不正	データ作成アプリ側で出力を見直す

9.3 開発担当者向け: E02・E04 の確定診断

- **再デプロイの徹底:** 環境変数 (`CLOSURES_PUBLISH_TOKEN` / `BLOB_READ_WRITE_TOKEN`) は **追加・変更後の再デプロイで初めて有効**になります。まず最新デプロイを Redeploy し、Ready を待つ。
- **Blob ストア接続の確認:** Vercel の **Storage** タブで Blob ストアが当該プロジェクトに Connect され、`BLOB_READ_WRITE_TOKEN` が入っているか (→ 12.2)。
- **生エラーの確認:** **Deployments** → **最新デプロイ** → **Runtime Logs** に、保存失敗時の 実際のエラー (`invalid token` / `store not found` など) が出ます。これで E04 の原因が一意に絞れます。

10. うまくいかないとき (トラブルシューティング)

症状	原因・対処
ホームに「通行止め・通行困難地点」ボタンが無い	? <code>closure=true</code> 付きで起動していない。MapGPS からのリンクで開き直す (同じタブのリロードでは維持される)
「マップに反映」が押せない	バージョンが未変更、または未入力。現在値と違う値に変更する
「【E01】公開トークンが正しくありません」と出る	公開トークンが誤り。もう一度「公開」を押して正しい値を入力
「公開に失敗しました」(E01~E05) と出る	メッセージ先頭の エラーコード で 対応が決まる → 9章 エラーコード別の対応
公開したのに他端末で変わらない	他端末が まだマップを開き直していない 。開き直すと反映される。バージョンを変え忘れていないかも確認
読み込みで「FeatureCollection 形式ではありません」等	geojson の形式不正。別アプリ側で出力を見直す

11. 参考: データの流れと仕組み

- **公開データの置き場**は Vercel Blob (公開ストア) です。「公開」 = ここへ全置換保存すること。
- 各端末は起動時に **公開API** `/api/closures` から最新を取得します。取れないとき (API 障害・オフライン等) は、最後に取得した内容 (端末内キャッシュ) を表示します。それも無い場合は表示なしです (古い情報を出すより安全)。

- **山中などオフライン**では、最後に取得した内容（端末内キャッシュ）を表示します。最新化は電波のある場所（登山口など）でマップを開いたときに行われます。
- git・GitHub・デプロイ待ちは、通常の公開フローには**登場しません**（設計書 §13）。

12. 付録: 公開トークンの決め方と Vercel 初回設定（設定担当者向け）

この章は **アプリを Vercel に用意する「設定担当者（開発担当）」**向けです。日常の更新・公開を行う運用担当者は、この章の作業は不要です（すでに設定済みの前提）。作業は原則 **初回に一度だけ**（＋トークンを変えたいときのローテーション時）です。

12.1 公開トークンの決め方

公開トークン（`CLOSURES_PUBLISH_TOKEN`）は、**意味のある文字列を考えて決めるのではなく、暗号的にランダムな値を生成して使います**。このトークン1個で公開データの全置換（偽情報の掲載・0 件公開による全消去）ができるため、**最重要の秘密**として扱います。

決め方の条件:

条件	内容
長くランダム	32 バイト（16 進で 64 文字）程度のランダム値。単語・日付・アプリ名などは使わない
埋め込まない	コードや git には入れない。Vercel の環境変数と、担当者のパスワードマネージャーだけに保存
交換できる	漏洩が疑われたら値を更新して無効化できる（→ 12.3）

生成コマンド（Windows PowerShell） — 暗号的に安全な乱数で 64 桁の 16 進文字列を作ります（`Get-Random` は暗号用途に不適なので使いません）：

```
$bytes = New-Object byte[] 32
[System.Security.Cryptography.RandomNumberGenerator]::Create().GetBytes($bytes)
-join ($bytes | ForEach-Object { '{0:x2}' -f $_ })
```

Node.js がある場合は 1 行でも作れます（同等の強度）：

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

生成した値は ①**Vercel の環境変数**、②**担当者のパスワードマネージャー** の 2 か所にだけ保存します。

12.2 Vercel での設定

必要なのは (A) **Blob ストア接続** と (B) **環境変数 `CLOSURES_PUBLISH_TOKEN`** の 2 つです。

(A) Blob ストアをプロジェクトに接続（初回のみ） これで公開データの保存に使う `BLOB_READ_WRITE_TOKEN` が**自動で**環境変数に追加されます（手動設定は不要）。

1. Vercel ダッシュボード → 対象プロジェクトを開く
2. 上部タブ **Storage** → **Create Database**（既存があれば選択） → **Blob**
3. ストアを作成し、対象プロジェクトへ **Connect**
4. Settings → Environment Variables に **BLOB_READ_WRITE_TOKEN** が入っていることを確認

(B) 環境変数 **CLOSURES_PUBLISH_TOKEN** を設定

1. プロジェクト → **Settings** → **Environment Variables**
2. **Add New** で入力
 - **Key:** **CLOSURES_PUBLISH_TOKEN**
 - **Value:** 12.1 で生成したランダム値
 - **Environment:** **Production**（プレビューでも公開テストするなら Preview も）
 - 「Sensitive（秘匿）」にできる場合はオンにする
3. **Save**

(C) 反映（再デプロイ） 環境変数は追加後の新しいデプロイから有効です。既存デプロイには反映されない
ので、**Deployments** → **最新デプロイの …** → **Redeploy** で再デプロイします。

動作確認

状態	挙動
トークン未設定のまま「公開」	API が 503 （トークン未設定）を返す
間違ったトークンで「公開」	401 「公開トークンが正しくありません」
正しいトークンで「公開」	200 成功（初回のみ端末でトークン入力）

12.3 トークンのローテーション（更新）

漏洩が疑われるとき、または定期更新のときに、公開トークン（**CLOSURES_PUBLISH_TOKEN**）を新しい値へ
入れ替えます。

全体の流れ: ① 新しい値を生成 → ② Vercel の環境変数を上書き → ③ 再デプロイ → ④ 各運用端末で新トークンを再入力。

① 新しい値を生成

12.1 のコマンドで新しいランダム値を作り、パスワードマネージャーに保存します（古い値は無効化される
ので上書きして構いません）。

② Vercel の環境変数を上書き（← 画面で迷いやすい箇所）

1. Vercel ダッシュボード → 対象プロジェクトを開く
2. 上部タブ **Settings** → 左メニュー **Environment Variables**
3. 一覧から **CLOSURES_PUBLISH_TOKEN** の行を探す
4. その行の右端にある「…」（三点メニュー）をクリック → **Edit**
 - ※「…」は行の右端にあり、見落としやすい場所です。ここが「既存の変数を編集する」入口です（新規の "Add New" ボタンではありません）。
5. 開いたパネルの **Value** を、① で作った新しい値に置き換え → **Save**
 - **Environment** は作成時と同じ **Production**（プレビューでも公開テストするなら Preview も）にチェックが入っていることを確認します。

「Sensitive (秘匿)」設定の変数を編集するときの注意 `CLOSURES_PUBLISH_TOKEN` は Sensitive のため、現在の値は画面に表示されません (●●●● 表示のまま／目のアイコンで開けない)。これは仕様です。古い値を確認する必要はなく、新しい値で上書きして **Save** するだけで更新できます。現在値が分からなくなった場合も、このローテーション手順でそのまま新しい値に置き換えれば復旧できます。

③ 再デプロイ (反映)

環境変数は追加・更新後の新しいデプロイから有効になります。既存の稼働中デプロイには反映されません。

1. 上部タブ **Deployments**
2. 最新デプロイの右端「…」 → **Redeploy**
3. 確認ダイアログで **Redeploy** を実行 ("Use existing Build Cache" はオンのままで構いません。環境変数の変更は反映されます)
4. ステータスが **Ready (緑)** になるまで待ちます (完了前に「公開」を押しても古い関数のままです)

④ 各運用端末で新トークンを再入力

更新後の初回「公開」時に、各端末で新しいトークンの入力を求められます。

- 古い値のまま公開すると 401 「公開トークンが正しくありません」となり、その端末の保存値は自動で消去されます。もう一度「公開」を押し、新しい値を入力してください。
- トークンが端末に保存されるのは公開が成功したときだけです。成功するまでは毎回入力を求められますが、正常な挙動です。

補足: `BLOB_READ_WRITE_TOKEN` を更新したいとき こちらは公開画面で入力するトークンではなく、サーバーが Blob ストアへ書き込むための鍵です。手動で編集せず、**Storage タブ (12.2(A))** の Blob ストア側で再発行・管理します。値を差し替えた場合も、反映には ③ の再デプロイが必要です。

なぜトークン強度とローテーションが重要かという背景は、[セキュリティレビュー closures-security-review-202607.md](#) の R1・R2 (最優先対応) を参照してください。